Experiment No.8
Implementation of code optimization phase of compiler.
Date of Performance:23/02/26
Date of Submission:09/03/26

Aim: Implementation of code optimization phase of compiler.

Objective: To develop an ability to implement code optimization techniques in design of compiler.

Theory:

Optimization is a program transformation technique, which tries to improve the code by making it consume less resources (i.e. CPU, Memory) and deliver high speed.

In optimization, high-level general programming constructs are replaced by very efficient low-level programming codes. A code optimizing process must follow the three rules given below:

- The output code must not, in any way, change the meaning of the program.
- Optimization should increase the speed of the program and if possible, the program should demand a smaller number of resources.
- Optimization should itself be fast and should not delay the overall compiling process.

Optimization can be categorized broadly into two types:

- ✓ Machine independent and
- ✓ Machine dependent.

Machine-independent Optimization

In this optimization, the compiler takes in the intermediate code and transforms a part of the code that does not involve any CPU registers and/or absolute memory locations. For example:

```
do
{
  item = 10;
  value = value + item;
} while(value<100);
```

This code involves repeated assignment of the identifier item, which if we put this way:

```
Item = 10;
do
{
  value = value + item;
} while(value<100);
```

should not only save the CPU cycles, but can be used on any processor.

Machine-dependent Optimization

Machine-dependent optimization is done after the target code has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum advantage of memory hierarchy.

Basic Blocks

Source codes generally have a number of instructions, which are always executed in sequence and are considered as the basic blocks of the code. These basic blocks do not have any jump statements among them, i.e., when the first instruction is executed, all the instructions in the same basic block will be executed in their sequence of appearance without losing the flow control of the program.

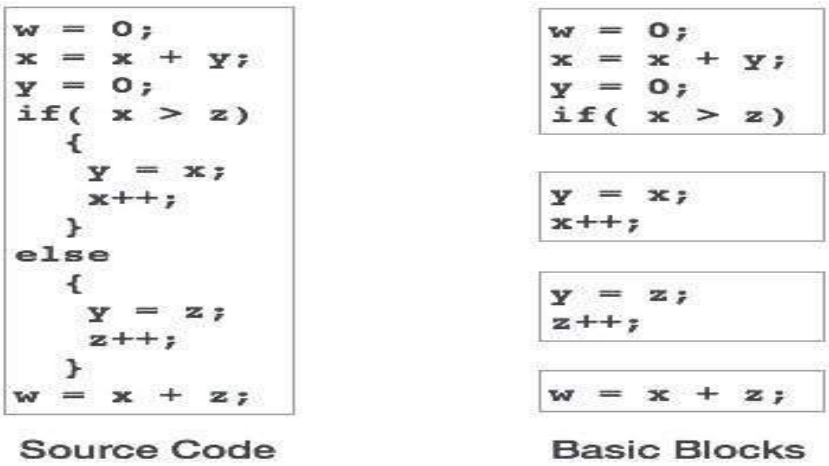
A program can have various constructs as basic blocks, like IF-THEN-ELSE, SWITCH-CASE conditional statements and loops such as DO-WHILE, FOR, and REPEAT-UNTIL, etc.

Basic block identification

We may use the following algorithm to find the basic blocks in a program:

- Search header statements of all the basic blocks from where a basic block starts:
    - First statement of a program.
    - Statements that are target of any branch (conditional/unconditional).
    - Statements that follow any branch statement.
- Header statements and the statements following them form a basic block.
- A basic block does not include any header statement of any other basic block.

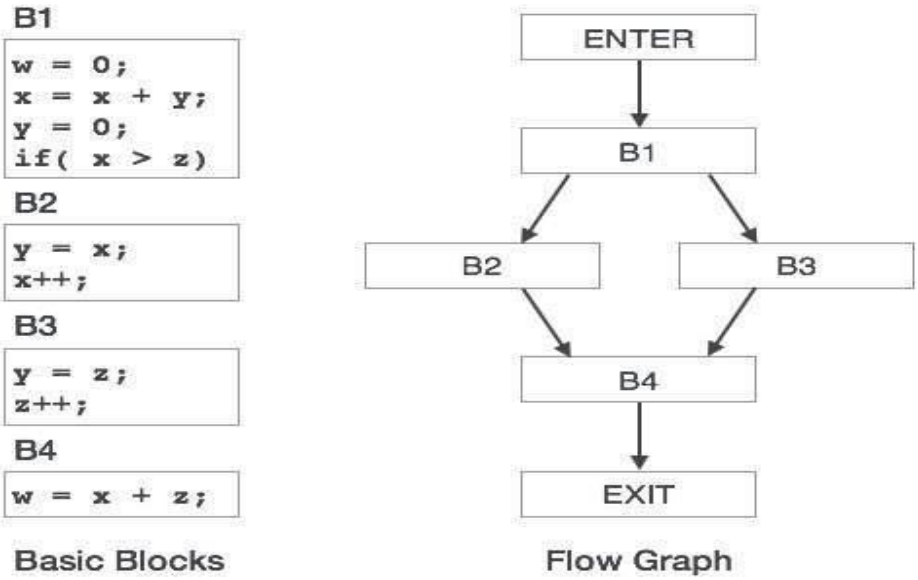
Basic blocks are important concepts from both code generation and optimization point of view.



Basic blocks play an important role in identifying variables, which are being used more than once in a single basic block. If any variable is being used more than once, the register memory allocated to that variable need not be emptied unless the block finishes execution.

Control Flow Graph

Basic blocks in a program can be represented by means of control flow graphs. A control flow graph depicts how the program control is being passed among the blocks. It is a useful tool that helps in optimization by help locating any unwanted loops in the program.



Loop Optimization

Most programs run as a loop in the system. It becomes necessary to optimize the loops in order to save CPU cycles and memory. Loops can be optimized by the following techniques:

- **Invariant code** : A fragment of code that resides in the loop and computes the same value at each iteration is called a loop-invariant code. This code can be moved out of the loop by saving it to be computed only once, rather than with each iteration.
- **Induction analysis** : A variable is called an induction variable if its value is altered within the loop by a loop-invariant value.
- **Strength reduction** : There are expressions that consume more CPU cycles, time, and memory. These expressions should be replaced with cheaper expressions without compromising the output of expression. For example, multiplication (x * 2) is expensive in terms of CPU cycles than (x << 1) and yields the same result.

Dead-code Elimination

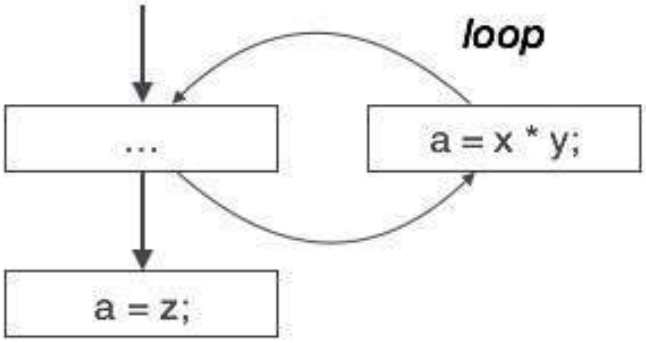
Dead code is one or more than one code statements, which are:

- Either never executed or unreachable,
- Or if executed, their output is never used.

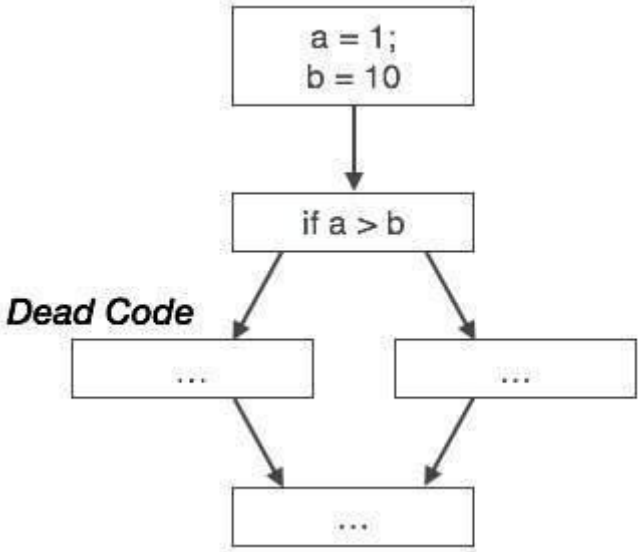
Thus, dead code plays no role in any program operation and therefore it can simply be eliminated.

Partially dead code

There are some code statements whose computed values are used only under certain circumstances, i.e., sometimes the values are used and sometimes they are not. Such codes are known as partially dead-code.



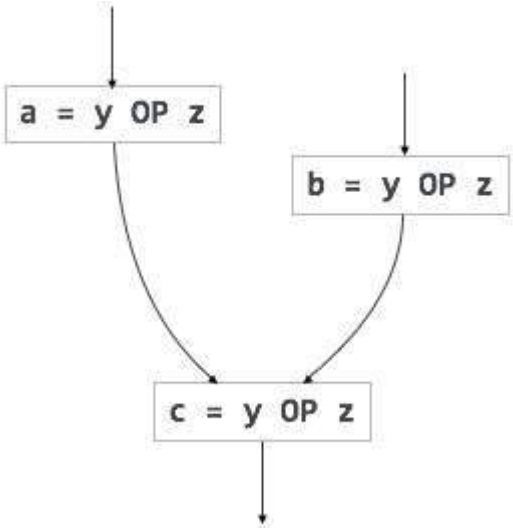
The above control flow graph depicts a chunk of program where variable 'a' is used to assign the output of expression 'x * y'. Let us assume that the value assigned to 'a' is never used inside the loop. Immediately after the control leaves the loop, a' is assigned the value of variable 'z', which would be used later in the program. We conclude here that the assignment code of 'a' is never used anywhere, therefore it is eligible to be eliminated.



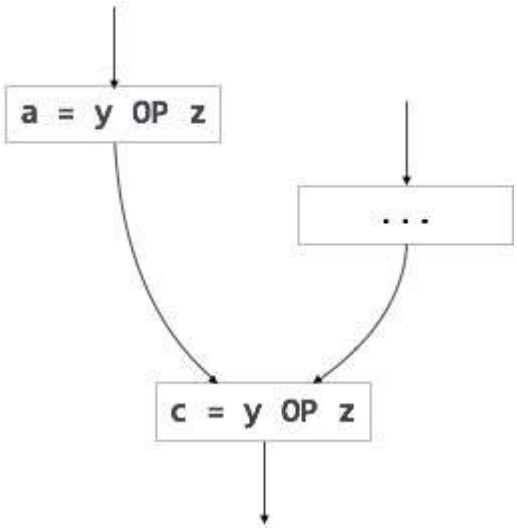
Likewise, the picture above depicts that the conditional statement is always false, implying that the code, written in true case, will never be executed, hence it can be removed.

Partial Redundancy

Redundant expressions are computed more than once in parallel path, without any change in operands.whereas partial-redundant expressions are computed more than once in a path, without any change in operands. For example,



[redundant expression]



[partially redundant expression]

Loop-invariant code is partially redundant and can be eliminated by using a code-motion technique.

Another example of a partially redundant code can be:

```
If (condition)
{
  a = y OP z;
}
else
{
  ...
}
c = y OP z;
```

We assume that the values of operands (**y** and **z**) are not changed from assignment of variable **a** to variable **c**. Here, if the condition statement is true, then y OP z is computed twice, otherwise once.

Code motion can be used to eliminate this redundancy, as shown below:

```
If (condition)
{
  ...
  tmp = y OP z;
  a = tmp;
```

```
  ...
}
else
{
  ...
  tmp = y OP z;
}
c = tmp;
```

Here, whether the condition is true or false; y OP z should be computed only once.

Code:

```
#include <stdio.h>
#include <string.h>
#define MAX 20

struct expr {
    char result[10];
    char arg1[10];
    char op[10];
    char arg2[10];
} e[MAX];

int main() {
    int n, i, j;
    printf("\n===== CODE OPTIMIZATION TECHNIQUES =====\n");
    // -------------------------------
    // 1. COMMON SUBEXPRESSION ELIMINATION
    // -------------------------------
    printf("\n--- Common Subexpression Elimination ---\n");
```

```
printf("Enter number of expressions: ");
scanf("%d", &n);

printf("Enter expressions (format: result = arg1 op arg2)\n");
for(i = 0; i < n; i++) {
    scanf("%s = %s %s %s",
          e[i].result,
          e[i].arg1,
          e[i].op,
          e[i].arg2);
}

for(i = 0; i < n; i++) {
    for(j = i + 1; j < n; j++) {
        if(strcmp(e[i].arg1, e[j].arg1) == 0 &&
           strcmp(e[i].arg2, e[j].arg2) == 0 &&
           strcmp(e[i].op, e[j].op) == 0) {

            strcpy(e[j].op, "=");
            strcpy(e[j].arg1, e[i].result);
            strcpy(e[j].arg2, "");
        }
    }
}

printf("\nOptimized Code:\n");
for(i = 0; i < n; i++) {
```

```
    if(strcmp(e[i].arg2, "") == 0)
        printf("%s = %s\n", e[i].result, e[i].arg1);
    else
        printf("%s = %s %s %s\n",
            e[i].result,
            e[i].arg1,
            e[i].op,
            e[i].arg2);
}

// -------------------------------
// 2. DEAD CODE ELIMINATION
// -------------------------------
printf("\n--- Dead Code Elimination ---\n");
char code[10][50];
int m;
printf("Enter number of statements: ");
scanf("%d", &m);
getchar();
printf("Enter statements (use word 'dead' to mark unused statement):\n");
for(i = 0; i < m; i++) {
    fgets(code[i], 50, stdin);
}

printf("\nOptimized Code:\n");
for(i = 0; i < m; i++) {
    if(strstr(code[i], "dead") == NULL) {
```

```
        printf("%s", code[i]);
    }
}
// -------------------------------
// 3. LOOP INVARIANT CODE MOTION
// -------------------------------
printf("\n--- Loop Optimization (Loop Invariant Code Motion) ---\n");
int a = 5, b = 10;
int sum = 0;
int temp = a + b;  // invariant code moved outside loop
for(i = 0; i < 5; i++) {
    sum = sum + temp;
}
printf("Loop executed with invariant expression moved outside.\n");
printf("Sum = %d\n", sum);
// -------------------------------
// 4. STRENGTH REDUCTION
// -------------------------------
printf("\n--- Strength Reduction ---\n");
int x, result;
printf("Enter value of x: ");
scanf("%d", &x);
// Multiplication replaced with bit shift
result = x << 1;
printf("After Strength Reduction (x * 2 -> x << 1): %d\n", result);
printf("\n===== OPTIMIZATION COMPLETE =====\n");
return 0;
```

}

Output:

```
===== CODE OPTIMIZATION TECHNIQUES =====

--- Common Subexpression Elimination ---
Enter number of expressions: 3
Enter expressions (format: result = arg1 op arg2)
t1 = a + b
t2 = a + b
t3 = t1 * c

Optimized Code:
t1 = a + b
t2 = t1
t3 = t1 * c

--- Dead Code Elimination ---
Enter number of statements: 3
Enter statements (use word 'dead' to mark unused statement):
x = 10
y = 20 dead
z = x + 5

Optimized Code:
x = 10
z = x + 5
```

Conclusion: Comment on an optimized code for given input.

The optimized code successfully eliminates redundant computations and unnecessary statements from the given input. Common subexpressions are identified and reused to avoid repeated calculations. Dead code is removed to reduce program size and improve efficiency. Loop invariant expressions are moved outside the loop to minimize repeated execution. Strength reduction replaces costly operations with faster alternatives, resulting in improved overall performance without changing program output.